Trace Market Intelligence - Intelligence Core Decisions

Status: Decision document. Nothing in this file is approved for build.
Product: Trace Market Intelligence (TraceMI/0.1)
Date: 24 August 2026
Related: Lattice Engine exploratory brief; current P0-P5 plan in docs/AI_GUIDE.md
This document compiles the proposed intelligence-core enhancements for individual accept or reject. It is not a build plan and does not change the running fixture app.

How to use this file

Reply with one of: include, reject, defer, or discuss next to each numbered item.
Do not treat this as an all-or-nothing package. Unmarked items stay out of the build.
Class / Meaning
KEEP / Current concept is appropriate
MODIFY / Current concept should change
EXPLORE FURTHER / Potentially valuable, not understood enough
DEFER / Useful later, premature now
REJECT / Complexity without enough value for the core
CANDIDATE FOR BUILD / Strong enough to consider after explicit approval

Context (short)

The Lattice Engine brief proposed nine modules between normalized data and the UI/AI. That sequence matches the product journey already in this repo:
Company -> Change -> Explanation -> Evidence -> Connections
The current plan already specifies entity/templates, deterministic metrics, explainable signals, peers, curated drivers, provenance, precompute on filing arrival, and AI only on top of structured output.
Lattice is therefore a strong hypothesis for P1, not a second platform. The recommended core, if named at all, is a claim compiler on gold packs - not nine services.
None of the items below are in the build until you say so.

1. Adopt "claim" as the intelligence unit

Class: MODIFY / candidate for P1 after gold measures exist
Effect: Merges Signal Engine and Anomaly Engine. Peers and drivers attach to claims instead of emitting a second alert stream.
What. Store a *claim*: a dated, typed assertion with a method (rule, historical percentile, peer comparison, interpretation) and an evidence chain. A "signal" becomes one kind of claim, not a separate ontology.
Why. Signals, anomalies, peer notes, and driver notes are the same kind of thing with different methods. Nine engines will double-count the same inventory gap.
Expected benefit. One evidence path, one ranker, no competing alert families.
Complexity. Moderate (schema + compiler). Not nine deployable services.
Risks. Vocabulary change versus the Lattice brief.
Decision: include / reject / defer / discuss

2. Evidence as required schema, not a module

Class: MODIFY (matches provenance rules already in docs/AI_GUIDE.md)
Effect: Deletes Evidence as a ninth engine that runs last.
What. Every claim, measure, and relationship edge must carry an evidence chain at write time. There is no later "evidence pass."
Why. A late evidence module will be skipped. If a calculation can exist without inputs, the design has already

failed.
Expected benefit. Auditability is structural. The user can always trace Company -> Change -> Evidence.
Complexity. Low if served numbers already store tag, accession, form, filed-at, mapping version, and confidence.
Risks. None if the compiler refuses claims without inputs.
Decision: include / reject / defer / discuss

3. Dossier ranker (modify Synthesis)

Class: CANDIDATE FOR BUILD in P1, after 10-20 claims work on the demo set
Effect: Synthesis becomes a rank/cluster step, not a narrator.
What. Cluster claims into a small number of *research cases* for the period (for example "working-capital deterioration") and rank them with a written deterministic policy (materiality, novelty, completeness of evidence).
Why. A flat list does not answer "what should I care about most." Fifty weak flags will bury three real ones.
Expected benefit. The change-first company page stays readable.
Complexity. Low if deterministic. High if an LLM writes a "primary issue."
Risks. If synthesis becomes a model-written headline with a fake High confidence, the product becomes a black box.
Decision: include / reject / defer / discuss

4. Historical unusualness as a claim method

Class: DEFER until measures exist for several years on the demo names
Effect: Replaces a standalone Anomaly Engine.
What. Attach history to the same claim: "this inventory-revenue gap is at the 97th percentile of this firm's own past," with the window and restatement policy in evidence.
Why. "Is it unusual?" is already in the UX sequence and is still empty. Unusualness is a method, not a sibling product that emits a second alert.
Expected benefit. Distinguishes a 12 pp gap that happens every year from one that does not.
Complexity. Needs enough history. as_of and fiscal alignment matter.
Risks. Bad windows, restatements, and mismatched periods will mint fake anomalies.
Decision: include / reject / defer / discuss

5. Competing explanations / contradiction among claims

Class: EXPLORE FURTHER / DEFER until claim quality is real
Effect: A small addition to the dossier, not a Hypothesis Engine.
What. Once claims exist, flag when two claims cannot both be true under a stated rule (margins down, peers flat, input costs down, MD&A blames inflation). List competing explanations. Do not pick a winner unless the evidence policy says so.
Why. Real research is disagreement. A single "primary issue" with High confidence is false precision.
Expected benefit. Fits "explanation" without pretending cause.
Complexity. Low after claims exist. High if a Hypothesis Engine is built first.
Risks. Noise if the underlying claims are junk.
Decision: include / reject / defer / discuss

6. Business-model templates and driver maps (not eight industry engines)

Class: KEEP what the plan already has; REJECT full industry Lattice packs in v1
Effect: Automotive / Banking "packs" become templates plus a small curated driver map, not forked products.
What. Share one measure/claim compiler. Vary the *allowed concepts and claims* by business-model template

(corporate, bank, ?) and attach a short economically plausible driver map. Do not ship eight complete industry engines.
Why. Inventory divergence is meaningless for a bank. You already proved this with HBT. Pack-per-industry will not be maintained.
Expected benefit. Same honesty, less fork cost.
Complexity. Already in the gated plan (corporate vs bank; 2-3 demo industries).
Risks. Too many templates too soon.
Decision: include / reject / defer / discuss

7. Relationship graph in the intelligence core

Class: DEFER (already later in the current plan)
Effect: Stays out of the core. Later it can be a data product with sourced, dated, material edges.
What. Do not put a knowledge graph or shock-propagation (TSMC -> NVDA -> MSFT) in the intelligence core for v0/v1. Industry, a curated peer set, and a few drivers are enough network until edges have source, confidence, and effective dates.
Why. The name "Lattice" tempts a graph-first build. Bad edges are worse than no graph.
Expected benefit of waiting. Graph work happens after claims and gold facts are trusted.
Complexity. High if done now; wasted if mapping is still wrong.
Risks. A pretty graph of garbage; implied causation.
Decision: include / reject / defer / discuss

8. Blended confidence formula and causal / graph-propagation core

Class: REJECT for this product's core
What not to do. Do not combine mapping quality, source independence, model score, and narrative agreement into one super-confidence number. Do not put causal graphs, instrumental variables, difference-in-differences, Granger tests, or multi-hop shock propagation in the company-page core.
Why. A perfect calculation can still support a weak story. Those parts of confidence are different and should stay visible. Causal and scenario machinery belongs in an isolated later lab (P5), with vintage data, or it becomes a credibility problem.
Expected benefit of rejecting. The product stays explainable.
Complexity if accepted. High, and it would fight the "no LLM arithmetic / no fake certainty" rules.
Risks. False precision; looking like an advice engine.
Decision: include / reject / defer / discuss
(Recommendation: reject.)

Explicitly not proposed

These stay out unless you add them later as new items:
- Nine deployable Lattice engines as the runtime
- An Evidence Engine that runs after the others
- An Anomaly Engine that emits a parallel alert stream
- AI inside the compiler (AI remains a researcher *over* claims)
- User-specific intelligence inside the core (personalization ranks *which* dossier to open, not the FY2025 math)
- Learning from dismissed signals in v1
- Pausing P0 warehouse work to build Lattice as a second system

Recommended stance (not an approval)

If an Intelligence Core is named at all, the version worth wanting is a claim compiler on gold packs - P1 done carefully - not a second platform.

Do not pause ingest, bronze, resolver, or shared packs for a nine-module Lattice build.

Response template

Copy and return:

```
1. Claim as the unit:          include / reject / defer / discuss
2. Evidence as schema:         include / reject / defer / discuss
3. Dossier ranker:             include / reject / defer / discuss
4. Historical unusualness:     include / reject / defer / discuss
5. Competing explanations:     include / reject / defer / discuss
6. Templates + driver maps:    include / reject / defer / discuss
7. Relationship graph in core: include / reject / defer / discuss
8. Blended confidence / causal core: include / reject / defer / discuss
```

Awaiting your marks. No item enters the build until you say so.